

datasets

April 24, 2025

1 Soil Organic Carbon Dataset Download

```
[1]: import ee

ee.Authenticate()
ee.Initialize()

[2]: import pandas as pd
import geopandas as gpd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import geemap as gm

sns.set_theme(style="whitegrid")
sns.set_context("talk")
```

2 Import Local Data

Get all data that is already downloaded.

2.1 Point Shapefile

```
[3]: path_to_soc_shp = './data/sichei_attr.shp'
path_to_soc_csv = './data/sichei_attr.csv'

# Load the shapefile
soc_gdf = gpd.read_file(path_to_soc_shp)

print(f'Columns GDF \n {soc_gdf.columns}')
print(f'Number of rows: {soc_gdf.shape[0]}')

soc = pd.read_csv(path_to_soc_csv)
print(f'Columns DF \n {soc.columns}')
print(f'Number of rows: {soc.shape[0]}')

soc.head()
```

<IPython.core.display.HTML object>

Columns GDF

```
Index(['Site', 'Cluster', 'Plot', 'Depthcode', 'predSOC', 'Date', 'Latitude',  
      'Longitude', 'Altitude', 'geometry'],  
      dtype='object')
```

Number of rows: 319

Columns DF

```
Index(['Site', 'Cluster', 'Plot', 'Depthcode', 'predSOC', 'Date', 'Latitude',  
      'Longitude', 'Altitude'],  
      dtype='object')
```

Number of rows: 319

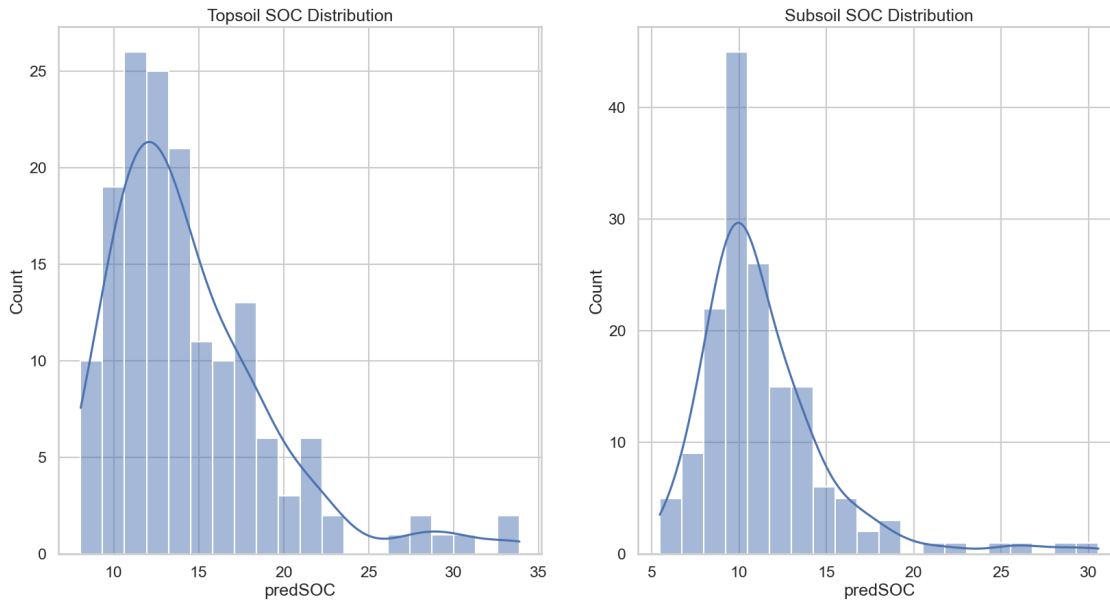
```
[3]:
```

	Site	Cluster	Plot	Depthcode	predSOC	Date	Latitude	Longitude	\
0	Sichei	1.0	1.0	Topsoil	11.246	10-Mar-21	0.711449	34.6406	
1	Sichei	1.0	1.0	Subsoil	8.619	10-Mar-21	0.711449	34.6406	
2	Sichei	2.0	1.0	Topsoil	11.516	11-Mar-21	0.731318	34.6339	
3	Sichei	2.0	1.0	Subsoil	9.866	11-Mar-21	0.731318	34.6339	
4	Sichei	3.0	1.0	Topsoil	14.464	12-Mar-21	0.768700	34.6443	

	Altitude
0	1544.81
1	1544.81
2	1552.42
3	1552.42
4	1611.33

```
[4]: # Split into top and subsoil  
topsoil = soc[soc['Depthcode'] == 'Topsoil']  
subsoil = soc[soc['Depthcode'] == 'Subsoil']  
  
# Plot the data  
fig, ax = plt.subplots(1, 2, figsize=(20, 10))  
  
sns.histplot(topsoil['predSOC'], bins=20, kde=True, ax=ax[0])  
ax[0].set_title('Topsoil SOC Distribution')  
  
sns.histplot(subsoil['predSOC'], bins=20, kde=True, ax=ax[1])  
ax[1].set_title('Subsoil SOC Distribution')  
  
plt.show()
```

<IPython.core.display.HTML object>



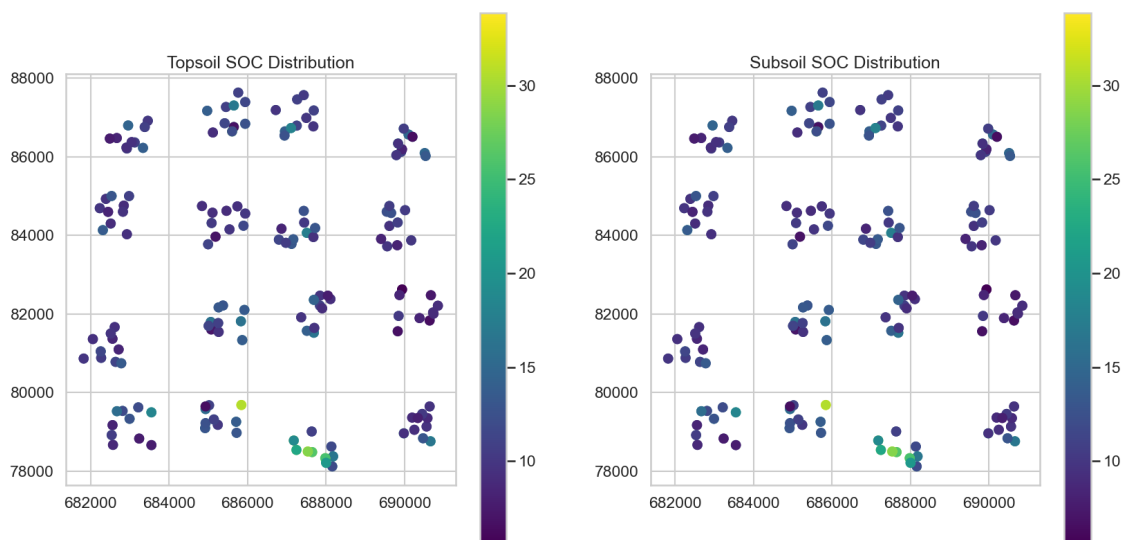
```
[5]: # Spatial Plot
fig, ax = plt.subplots(1, 2, figsize=(20, 10))

soc_gdf.plot(column='predSOC', cmap='viridis', legend=True, ax=ax[0])
ax[0].set_title('Topsoil SOC Distribution')

soc_gdf.plot(column='predSOC', cmap='viridis', legend=True, ax=ax[1])
ax[1].set_title('Subsoil SOC Distribution')

plt.show()
```

<IPython.core.display.HTML object>



```
[6]: soc_gdf.head()
```

<IPython.core.display.HTML object>

```
[6]:
```

	Site	Cluster	Plot	Depthcode	predSOC	Date	Latitude	Longitude	\
0	Sichei	1.0	1.0	Topsoil	11.246	10-Mar-21	0.711449	34.6406	
1	Sichei	1.0	1.0	Subsoil	8.619	10-Mar-21	0.711449	34.6406	
2	Sichei	2.0	1.0	Topsoil	11.516	11-Mar-21	0.731318	34.6339	
3	Sichei	2.0	1.0	Subsoil	9.866	11-Mar-21	0.731318	34.6339	
4	Sichei	3.0	1.0	Topsoil	14.464	12-Mar-21	0.768700	34.6443	

	Altitude	geometry
0	1544.81	POINT (682568.835 78668.993)
1	1544.81	POINT (682568.835 78668.993)
2	1552.42	POINT (681822.254 80865.753)
3	1552.42	POINT (681822.254 80865.753)
4	1611.33	POINT (682978.344 84999.73)

```
[7]: # Merge the topsoil and subsoil data
soc_gdf = soc_gdf.pivot_table(index='geometry', values='predSOC',
    ↪aggfunc='mean').reset_index()
soc_gdf = gpd.GeoDataFrame(soc_gdf, geometry='geometry')
soc_gdf.head()
```

<IPython.core.display.HTML object>

```
[7]:
```

	geometry	predSOC
0	POINT (682568.835 78668.993)	9.9325
1	POINT (682535.352 78918.883)	12.5035
2	POINT (682557.524 79171.778)	9.4790
3	POINT (682668.709 79526.324)	16.8675
4	POINT (682813.413 79533.121)	15.0005

2.2 The Bounding of the Data

Since I shall be getting data from Google Earth Engine using the python API, it is important to describe the bounding box of the dataset. This will be achieved by describing the extent of the data then, providing a buffer of 2km around the extent. The resulting polygon will then be exported as the bbox of the region of study

```
[8]: from shapely.geometry import box
xmin, ymin, xmax, ymax = soc_gdf.total_bounds
bbox = box(xmin, ymin, xmax, ymax)
bbox_gdf = gpd.GeoDataFrame(geometry=[bbox], crs=soc_gdf.crs)
buffered_bbox_gdf = bbox_gdf.buffer(1000)
```

```
buffered_bbox_gdf = gpd.GeoDataFrame(geometry=buffered_bbox_gdf, crs=soc_gdf.
↳ crs)

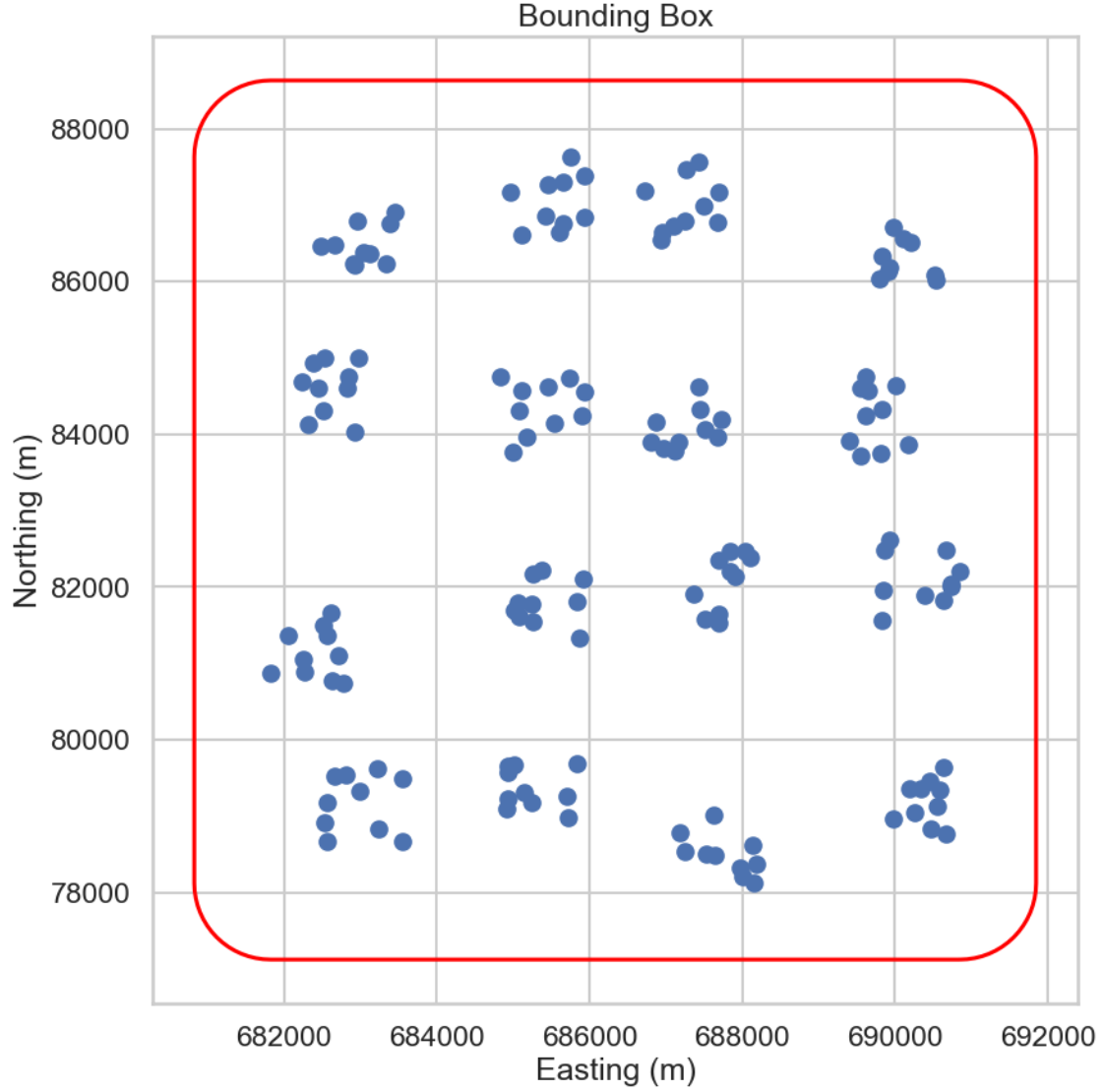
# COnvert the bounding box to ee.Geometry
bbox_ee = gm.geopandas_to_ee(buffered_bbox_gdf)

buffered_bbox_gdf.to_file("./data/bbox.shp", driver='ESRI Shapefile')
```

<IPython.core.display.HTML object>

```
[9]: # Plot the bbox
fig, ax = plt.subplots(1, 1, figsize=(10, 10), dpi=100)
soc_gdf.plot(ax=ax)
buffered_bbox_gdf.boundary.plot(ax=ax, color='red')
plt.title('Bounding Box')
plt.xlabel('Easting (m)')
plt.ylabel('Northing (m)')
plt.show()
```

<IPython.core.display.HTML object>



3 Datasets

Data from Google Earth Engine for Predicting Soil Organic Carbon.

The following datasets will be used in the study: - Sentinel-1 (and Derivatives) - Sentinel-2 (and Derivatives) - Land Use Land Cover - WAPOR Evapotranspiration - Distance to Roads - Distance to Rivers - Lithology and Soil Types - DEM and Derivatives (TWI, Curvature, Slope, Stream Power Index)

3.0.1 Sentinel 1 RADAR Data

Collection of SAR data from the COPERNICUS collection

```
[10]: def mask_edge(image):
    edge = image.lt(-30.0)
    masked_image = image.mask().And(edge.Not())
    return image.updateMask(masked_image)
```

<IPython.core.display.HTML object>

```
[11]: img_vv_vh = (
    ee.ImageCollection('COPERNICUS/S1_GRD')
    .filter(ee.Filter.listContains('transmitterReceiverPolarisation', 'VV'))
    .filter(ee.Filter.eq('instrumentMode', 'IW'))
    .select(['VV', 'VH'])
    .map(mask_edge)
)

focus_date = ee.Filter.date('2021-03-01', '2021-03-20')
img_mean = img_vv_vh.filter(focus_date).mean().clip(bbox_ee)
```

<IPython.core.display.HTML object>

3.0.2 Sentinel 1 SAR Derivatives

The indices of sentinel-1 can be used to give insight into subsurface properties. The following indices will be computed: - VH/VV Ratio (VVVHR) > This index helps to understand soil moisture and surface roughness, which are correlated with SOC.

$$VVVHR = \frac{VV}{VH}$$

- Radar Vegetation Index

$$RVI = \frac{4 * VH}{VV + VH}$$

- Normalized Difference Index

$$NDI = \frac{VV - VH}{VV + VH}$$

- Cross-Polarization Ratio

$$CPR = \frac{VH}{VV}$$

```
[12]: def compute_vvvhr(image):
    """Computes VH/VV Ratio Index for Sentinel-1 Image."""
    vh = image.select('VH')
    vv = image.select('VV')
    vvvhr = vh.divide(vv).rename('VVVHR')
    return image.addBands(vvvhr)

def compute_rvi(image):
    """Computes Radar Vegetation Index (RVI) for Sentinel-1 Image."""
    vh = image.select('VH')
    vv = image.select('VV')
```

```

    rvi = (vh.multiply(4)).divide(vv.add(vh)).rename('RVI')
    return image.addBands(rvi)

def compute_ndi(image):
    """Computes Normalized Difference Index (NDI) for Sentinel-1 Image."""
    vh = image.select('VH')
    vv = image.select('VV')
    ndi = vv.subtract(vh).divide(vv.add(vh)).rename('NDI')
    return image.addBands(ndi)

def compute_cpr(image):
    """Computes Cross-Polarization Ratio (CPR) for Sentinel-1 Image."""
    vh = image.select('VH')
    vv = image.select('VV')
    cpr = vh.divide(vv).rename('CPR')
    return image.addBands(cpr)

```

<IPython.core.display.HTML object>

```

[13]: # apply the functions to the image
img_mean = compute_vvvhrr(img_mean)
img_mean = compute_rvi(img_mean)
img_mean = compute_ndi(img_mean)
img_mean = compute_cpr(img_mean)

# Get the image
img_mean.getInfo()

```

<IPython.core.display.HTML object>

```

[13]: {'type': 'Image',
      'bands': [{'id': 'VV',
                  'data_type': {'type': 'PixelType', 'precision': 'double'},
                  'crs': 'EPSG:4326',
                  'crs_transform': [1, 0, 0, 0, 1, 0]},
                 {'id': 'VH',
                  'data_type': {'type': 'PixelType', 'precision': 'double'},
                  'crs': 'EPSG:4326',
                  'crs_transform': [1, 0, 0, 0, 1, 0]},
                 {'id': 'VVVHR',
                  'data_type': {'type': 'PixelType', 'precision': 'double'},
                  'crs': 'EPSG:4326',
                  'crs_transform': [1, 0, 0, 0, 1, 0]},
                 {'id': 'RVI',
                  'data_type': {'type': 'PixelType', 'precision': 'double'},
                  'crs': 'EPSG:4326',
                  'crs_transform': [1, 0, 0, 0, 1, 0]},
                 {'id': 'NDI',

```

```

        'data_type': {'type': 'PixelType', 'precision': 'double'},
        'crs': 'EPSG:4326',
        'crs_transform': [1, 0, 0, 0, 1, 0]},
    {'id': 'CPR',
     'data_type': {'type': 'PixelType', 'precision': 'double'},
     'crs': 'EPSG:4326',
     'crs_transform': [1, 0, 0, 0, 1, 0]}}}

```

```

[14]: # Convert FeatureCollection to Geometry if needed
bbox_ee = bbox_ee.geometry() if isinstance(bbox_ee, ee.FeatureCollection) else
↳ bbox_ee

# Export the Image to local directory
filename = './exports/sentinell1_indices.tif'

gm.ee_export_image(img_mean, filename=filename, scale=20, region=bbox_ee,
↳ file_per_band=False)

```

<IPython.core.display.HTML object>

Generating URL ...

Downloading data from <https://earthengine.googleapis.com/v1/projects/152882661905/thumbnails/c74177b2ae517eba028ec4ea2677b626-b916acfb287d9213df346f89efb98478:getPixels>

Please wait ...

Data downloaded to c:\Users\roywaswa\PROJECTS\Data-Science\02-SOC\exports\sentinell1_indices.tif

```

[15]: sarMap = gm.Map()
sarMap.center_object(bbox_ee, 12)
sarMap.add_layer(img_mean, {'min': -25, 'max': 5}, 'Multi-T Mean ASC', True)
sarMap

```

<IPython.core.display.HTML object>

```

[15]: Map(center=[0.7494919277365318, 34.67445436967993],
controls=(WidgetControl(options=['position', 'transparent_...

```

```

[16]: # Sample the data from the image using the points from the shapefile
soc_points = gm.geopandas_to_ee(soc_gdf)

socsoil_points = img_mean.sampleRegions(
    collection=soc_points,
    scale=10,
    geometries=True
)

```

<IPython.core.display.HTML object>

3.0.3 Sentinel 2 MSI Data

```
[17]: def mask_s2_clouds(image):  
    """Masks clouds in a Sentinel-2 image using the QA band.  
  
    Args:  
        image (ee.Image): A Sentinel-2 image.  
  
    Returns:  
        ee.Image: A cloud-masked Sentinel-2 image.  
    """  
    qa = image.select('QA60')  
  
    # Bits 10 and 11 are clouds and cirrus, respectively.  
    cloud_bit_mask = 1 << 10  
    cirrus_bit_mask = 1 << 11  
  
    # Both flags should be set to zero, indicating clear conditions.  
    mask = (  
        qa.bitwiseAnd(cloud_bit_mask)  
        .eq(0)  
        .And(qa.bitwiseAnd(cirrus_bit_mask).eq(0))  
    )  
  
    return image.updateMask(mask).divide(10000)  
  
dataset = (  
    ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')  
    .filterDate('2021-03-01', '2021-04-01')  
    # Pre-filter to get less cloudy granules.  
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))  
    .map(mask_s2_clouds)  
    .select(['B2', 'B3', 'B4', 'B5', 'B6', 'B7', 'B8', 'B8A', 'B9', 'B11', 'B12'])  
    .mean().clip(bbox_ee)  
)
```

<IPython.core.display.HTML object>

3.0.4 Sentinel-2 Indices for Soil Organic Carbon Estimation

Sentinel-2 provides multispectral imagery that is useful for estimating Soil Organic Carbon (SOC). Several vegetation and soil indices derived from Sentinel-2 bands can be correlated with SOC.

1. Normalized Difference Vegetation Index (NDVI) NDVI is commonly used to assess vegetation health and biomass, both of which are linked to SOC.

$$NDVI = \frac{B8 - B4}{B8 + B4}$$

2. Normalized Difference Soil Index (NDSI) NDSI helps distinguish bare soil from vegetation and can be useful in SOC estimation.

$$NDSI = \frac{B11 - B4}{B11 + B4}$$

3. Soil Adjusted Vegetation Index (SAVI) SAVI accounts for soil brightness and is useful in semi-arid and arid regions.

$$SAVI = \frac{(B8 - B4) \times (1 + L)}{B8 + B4 + L}$$

where (L) is a soil brightness correction factor (typically 0.5).

4. Bare Soil Index (BSI) BSI helps in detecting bare soil areas, which are important for SOC estimation.

$$BSI = \frac{(B11 + B4) - (B8 + B2)}{(B11 + B4) + (B8 + B2)}$$

5. Normalized Burn Ratio (NBR) NBR is useful for assessing soil conditions after disturbances such as wildfires, which influence SOC.

$$NBR = \frac{B8 - B12}{B8 + B12}$$

These indices provide valuable insights into vegetation, soil properties, and moisture levels, all of which are essential for estimating Soil Organic Carbon (SOC). They can be used in combination with machine learning models or regression analyses for SOC prediction.

```
[18]: import ee

def compute_ndvi(image):
    """Computes NDVI for Sentinel-2 Image."""
    ndvi = image.normalizedDifference(['B8', 'B4']).rename('NDVI')
    return image.addBands(ndvi)

def compute_ndsi(image):
    """Computes NDSI for Sentinel-2 Image."""
    ndsi = image.normalizedDifference(['B11', 'B4']).rename('NDSI')
    return image.addBands(ndsi)

def compute_savi(image, L=0.5):
    """Computes SAVI for Sentinel-2 Image."""
    nir = image.select('B8')
    red = image.select('B4')
    savi = ((nir.subtract(red)).multiply(1 + L)).divide(nir.add(red).add(L)).
    ↪rename('SAVI'))
```

```

    return image.addBands(savi)

def compute_bsi(image):
    """Computes BSI for Sentinel-2 Image."""
    swir = image.select('B11')
    red = image.select('B4')
    nir = image.select('B8')
    blue = image.select('B2')
    bsi = ((swir.add(red)).subtract(nir.add(blue))).divide((swir.add(red)).
    ↪ add(nir.add(blue))).rename('BSI')
    return image.addBands(bsi)

def compute_nbr(image):
    """Computes NBR for Sentinel-2 Image."""
    nbr = image.normalizedDifference(['B8', 'B12']).rename('NBR')
    return image.addBands(nbr)

def apply_all_indices(image):
    """Applies all Sentinel-2 indices to the given ee.Image."""
    image = compute_ndvi(image)
    image = compute_ndsi(image)
    image = compute_savi(image)
    image = compute_bsi(image)
    image = compute_nbr(image)
    return image

```

<IPython.core.display.HTML object>

```

[ ]: # Apply the functions to the image
dataset = apply_all_indices(dataset)

# dataset.getInfo()

```

<IPython.core.display.HTML object>

```

[ ]: {'type': 'Image',
      'bands': [{'id': 'B2',
                  'data_type': {'type': 'PixelType',
                                'precision': 'float',
                                'min': 0,
                                'max': 6.553500175476074},
                  'dimensions': [2, 2],
                  'origin': [34, 0],
                  'crs': 'EPSG:4326',
                  'crs_transform': [1, 0, 0, 0, 1, 0]},
                {'id': 'B3',
                  'data_type': {'type': 'PixelType',
                                'precision': 'float',

```

```

    'min': 0,
    'max': 6.553500175476074},
    'dimensions': [2, 2],
    'origin': [34, 0],
    'crs': 'EPSG:4326',
    'crs_transform': [1, 0, 0, 0, 1, 0]},
    {'id': 'B4',
     'data_type': {'type': 'PixelType',
                    'precision': 'float',
                    'min': 0,
                    'max': 6.553500175476074},
     'dimensions': [2, 2],
     'origin': [34, 0],
     'crs': 'EPSG:4326',
     'crs_transform': [1, 0, 0, 0, 1, 0]},
    {'id': 'B5',
     'data_type': {'type': 'PixelType',
                    'precision': 'float',
                    'min': 0,
                    'max': 6.553500175476074},
     'dimensions': [2, 2],
     'origin': [34, 0],
     'crs': 'EPSG:4326',
     'crs_transform': [1, 0, 0, 0, 1, 0]},
    {'id': 'B6',
     'data_type': {'type': 'PixelType',
                    'precision': 'float',
                    'min': 0,
                    'max': 6.553500175476074},
     'dimensions': [2, 2],
     'origin': [34, 0],
     'crs': 'EPSG:4326',
     'crs_transform': [1, 0, 0, 0, 1, 0]},
    {'id': 'B7',
     'data_type': {'type': 'PixelType',
                    'precision': 'float',
                    'min': 0,
                    'max': 6.553500175476074},
     'dimensions': [2, 2],
     'origin': [34, 0],
     'crs': 'EPSG:4326',
     'crs_transform': [1, 0, 0, 0, 1, 0]},
    {'id': 'B8',
     'data_type': {'type': 'PixelType',
                    'precision': 'float',
                    'min': 0,
                    'max': 6.553500175476074},

```

```

    'dimensions': [2, 2],
    'origin': [34, 0],
    'crs': 'EPSG:4326',
    'crs_transform': [1, 0, 0, 0, 1, 0]],
    {'id': 'B8A',
     'data_type': {'type': 'PixelType',
                    'precision': 'float',
                    'min': 0,
                    'max': 6.553500175476074},
     'dimensions': [2, 2],
     'origin': [34, 0],
     'crs': 'EPSG:4326',
     'crs_transform': [1, 0, 0, 0, 1, 0]],
    {'id': 'B9',
     'data_type': {'type': 'PixelType',
                    'precision': 'float',
                    'min': 0,
                    'max': 6.553500175476074},
     'dimensions': [2, 2],
     'origin': [34, 0],
     'crs': 'EPSG:4326',
     'crs_transform': [1, 0, 0, 0, 1, 0]],
    {'id': 'B11',
     'data_type': {'type': 'PixelType',
                    'precision': 'float',
                    'min': 0,
                    'max': 6.553500175476074},
     'dimensions': [2, 2],
     'origin': [34, 0],
     'crs': 'EPSG:4326',
     'crs_transform': [1, 0, 0, 0, 1, 0]],
    {'id': 'B12',
     'data_type': {'type': 'PixelType',
                    'precision': 'float',
                    'min': 0,
                    'max': 6.553500175476074},
     'dimensions': [2, 2],
     'origin': [34, 0],
     'crs': 'EPSG:4326',
     'crs_transform': [1, 0, 0, 0, 1, 0]],
    {'id': 'NDVI',
     'data_type': {'type': 'PixelType',
                    'precision': 'float',
                    'min': -1,
                    'max': 1},
     'dimensions': [2, 2],
     'origin': [34, 0],

```

```

'crs': 'EPSG:4326',
'crs_transform': [1, 0, 0, 0, 1, 0]],
{'id': 'NDSI',
 'data_type': {'type': 'PixelType',
 'precision': 'float',
 'min': -1,
 'max': 1},
 'dimensions': [2, 2],
 'origin': [34, 0],
 'crs': 'EPSG:4326',
 'crs_transform': [1, 0, 0, 0, 1, 0]],
{'id': 'SAVI',
 'data_type': {'type': 'PixelType',
 'precision': 'double',
 'min': -19.660500526428223,
 'max': 19.660500526428223},
 'dimensions': [2, 2],
 'origin': [34, 0],
 'crs': 'EPSG:4326',
 'crs_transform': [1, 0, 0, 0, 1, 0]],
{'id': 'BSI',
 'data_type': {'type': 'PixelType', 'precision': 'float'},
 'dimensions': [2, 2],
 'origin': [34, 0],
 'crs': 'EPSG:4326',
 'crs_transform': [1, 0, 0, 0, 1, 0]],
{'id': 'NBR',
 'data_type': {'type': 'PixelType',
 'precision': 'float',
 'min': -1,
 'max': 1},
 'dimensions': [2, 2],
 'origin': [34, 0],
 'crs': 'EPSG:4326',
 'crs_transform': [1, 0, 0, 0, 1, 0]]],
'properties': {'system:footprint': {'type': 'Polygon',
 'coordinates': [[[34.71498279149133, 0.697430854797614],
 [34.715863329929334, 0.697474077227798],
 [34.71673541948374, 0.697603974274401],
 [34.71759066153632, 0.697819294849147],
 [34.71842081973766, 0.698117965201754],
 [34.71921789932396, 0.698497108894974],
 [34.71997422410658, 0.698953074509295],
 [34.72068251039326, 0.699481470810384],
 [34.721335937129076, 0.700077209040496],
 [34.72192821158194, 0.700734551926502],
 [34.72245362994025, 0.701447168932537],

```

[34.722907132239264, 0.70220819722518],
[34.723284351087365, 0.703010307764075],
[34.72358165372325, 0.703845775881595],
[34.7237961769989, 0.70470655567194],
[34.72392585495177, 0.705584357473415],
[34.72396943870055, 0.706470727697833],
[34.72400307463378, 0.792474439884862],
[34.72396018335415, 0.79336084402769],
[34.72383118941229, 0.794238747982815],
[34.72361733500262, 0.795099697163396],
[34.72332067955452, 0.795935400252286],
[34.722944079902874, 0.796737809048306],
[34.722491162779875, 0.797499195971875],
[34.721966289892556, 0.798212228483672],
[34.72137451592231, 0.798870039699733],
[34.72072153985061, 0.799466294522968],
[34.72001365007956, 0.799995250654219],
[34.719257663875595, 0.80045181389527],
[34.71846086171926, 0.800831587211159],
[34.717630917193276, 0.801130913079182],
[34.716775823083985, 0.80134690871663],
[34.71590381440766, 0.801477493847859],
[34.715023289103065, 0.801521410743092],
[34.633926622820844, 0.801554795382896],
[34.63304602504929, 0.801511601626473],
[34.63217387306872, 0.801381728807391],
[34.63131856626598, 0.801166427570022],
[34.63048834181606, 0.80086777129478],
[34.62969119534962, 0.800488636133717],
[34.62893480394625, 0.800032673314301],
[34.62822645219526, 0.799504273978053],
[34.627572962036176, 0.798908526892567],
[34.626980627055055, 0.798251169444125],
[34.626455151869145, 0.797538532382891],
[34.62600159718431, 0.79677747885282],
[34.625624331054105, 0.795975338293542],
[34.62532698681021, 0.79513983585084],
[34.62511242806949, 0.794279017975686],
[34.624982721154744, 0.793401174928466],
[34.62493911519466, 0.792514760934901],
[34.62490740989932, 0.706506672432672],
[34.624950361465146, 0.705620225751969],
[34.625079418497975, 0.704742285679834],
[34.62529333802836, 0.703881307361515],
[34.625590059797794, 0.703045582578408],
[34.62596672610383, 0.702243159890247],
[34.626419709325496, 0.701481767120275],

```
[34.62694464686375, 0.700768736930014],
[34.627536483160405, 0.700110936200454],
[34.628189518390634, 0.699514699899825],
[34.62889746335981, 0.698985770074849],
[34.62965350007612, 0.698529240553003],
[34.63045034741508, 0.698149507888297],
[34.631280331243836, 0.697850229022898],
[34.63213545832939, 0.697634286072251],
[34.6330074933192, 0.697503758572685],
[34.633888038052596, 0.697459903458639],
[34.71498279149133, 0.697430854797614]]]]}}}
```

```
[20]: # Convert FeatureCollection to Geometry if needed
bbox_ee = bbox_ee.geometry() if isinstance(bbox_ee, ee.FeatureCollection) else
↳ bbox_ee

# Export the Image to local directory
filename = './exports/sentinel2_indices.tif'

gm.ee_export_image(dataset, filename=filename, scale=20, region=bbox_ee,
↳ file_per_band=False)
```

<IPython.core.display.HTML object>

Generating URL ...

Downloading data from <https://earthengine.googleapis.com/v1/projects/152882661905/thumbnails/9360f5a3776141d2fc915bb5d5e48a7c-e3c8dfd9b7763e088bf3be15bfb4385e:getPixels>

Please wait ...

Data downloaded to c:\Users\roywaswa\PROJECTS\Data-Science\02-SOC\exports\sentinel2_indices.tif

```
[21]: visualization = {
    'min': 0.0,
    'max': 0.3,
    'bands': ['B4', 'B3', 'B2'],
}

m = gm.Map()
m.center_object(bbox_ee, 12)
m.add_layer(dataset, visualization, 'RGB')
m
```

<IPython.core.display.HTML object>

```
[21]: Map(center=[0.7494919277365318, 34.67445436967993],
controls=(WidgetControl(options=['position', 'transparent_...
```

```
[ ]: # Sample the data from the image using the points from the shapefile
socsoil_points = dataset.sampleRegions(
    collection=socsoil_points,
    scale=10,
    geometries=True
)
```

<IPython.core.display.HTML object>

```
[ ]:
      geometry      B11      B12      B2      B3      B4 \
0 POINT (34.64061 0.71142) 0.290867 0.197833 0.048367 0.081767 0.088233
1 POINT (34.64034 0.71367) 0.323333 0.249733 0.049567 0.078367 0.094800
2 POINT (34.64052 0.716) 0.288000 0.217133 0.054700 0.087900 0.095100
3 POINT (34.64151 0.71924) 0.284400 0.188667 0.040300 0.069233 0.070267
4 POINT (34.64277 0.71924) 0.297900 0.189567 0.048200 0.078833 0.078367

      B5      B6      B7      B8 ...      NBR      NDI      NDSI \
0 0.136800 0.224033 0.253400 0.250300 ... 0.117078 -0.165985 0.534512
1 0.159800 0.224500 0.256033 0.256167 ... 0.012717 -0.222956 0.546556
2 0.136300 0.239200 0.272267 0.272800 ... 0.113621 -0.398032 0.503524
3 0.133900 0.223500 0.260400 0.264833 ... 0.167953 -0.136337 0.603759
4 0.132867 0.234000 0.268400 0.264000 ... 0.164107 -0.180343 0.583451

      NDVI      RVI      SAVI      VH      VV      VVHR      predSOC
0 0.478732 2.331970 0.289911 -17.836760 -12.758419 1.398038 9.9325
1 0.459778 2.445912 0.284441 -15.882943 -10.091735 1.573857 12.5035
2 0.483012 2.796063 0.307121 -19.025523 -8.192064 2.322434 9.4790
3 0.580623 2.272675 0.349479 -14.729759 -11.195216 1.315719 16.8675
4 0.542206 2.360686 0.330557 -16.927974 -11.755173 1.440045 15.0005
```

[5 rows x 24 columns]

```
[44]: soilpoints = gm.ee_to_gdf(socsoil_points)
      # soilpoints.head()
```

<IPython.core.display.HTML object>

```
[35]: soilpoints.to_file("./exports/sichei_socpoints.shp", driver='ESRI Shapefile')
```

<IPython.core.display.HTML object>

3.0.5 Raster Datasets

Obtaining the rest of the data from local raster files: - aspect - slope - curvature - twi - strm - distance from rivers - distance from roads - soil types (SOTER)

```
[87]: import rasterio
      import os
      import numpy as np
```

```

import pandas as pd
from rasterio.mask import mask
from shapely.geometry import mapping

raster_files = [
    './rasters/aspect_sichei.tif',
    './rasters/slope_sichei.tif',
    './rasters/curvature_sichei.tif',
    './rasters/model_twi.tif',
    './rasters/srtm_30.tif',
    './rasters/rivers_distance_sichei.tif',
    './rasters/roads_distances_sichei.tif',
    # './rasters/soterv2_sichei.tif',
]

```

<IPython.core.display.HTML object>

```

[ ]: with rasterio.open(raster_files[0]) as src:
    # Read the data and mask it with the shapefile geometry
    # get crs
    crs = src.crs
    print(f'CRS: {crs}')

soc_utm = soilpoints.to_crs(crs.to_epsg())

```

<IPython.core.display.HTML object>

CRS: EPSG:32636

```

[67]: def extract_raster_values(point, raster_path):
    with rasterio.open(raster_path) as src:
        try:
            value = next(src.sample([(point.x, point.y)]))[0]
            return value
        except IndexError:
            return np.nan # Handle cases where point might be outside raster_
↪ extent

for raster_file in raster_files:
    get_first_name = lambda path: os.path.splitext(os.path.basename(path))[0]
    column_name = get_first_name(raster_file)
    soc_utm[column_name] = soc_utm.apply(lambda row: extract_raster_values(row.
↪ geometry, raster_file), axis=1)

# If your SOTERv2_Sichei is a raster with categorical soil types:
# You might need to extract the soil type at each point as well.
# Then, you'll likely need to handle categorical variables (e.g., one-hot_
↪ encoding) later.

```

```
# If it's a vector layer, you'd do a spatial join.
```

```
soc_utm.head()
```

```
<IPython.core.display.HTML object>
```

```
[67]:
```

	geometry	B11	B12	B2	B3	\
0	POINT (682569.883 78665.874)	0.290867	0.197833	0.048367	0.081767	
1	POINT (682539.797 78914.194)	0.323333	0.249733	0.049567	0.078367	
2	POINT (682559.703 79172.464)	0.288000	0.217133	0.054700	0.087900	
3	POINT (682669.568 79530.099)	0.284400	0.188667	0.040300	0.069233	
4	POINT (682809.559 79530.149)	0.297900	0.189567	0.048200	0.078833	

	B4	B5	B6	B7	B8	...	VV	VVVHR	\
0	0.088233	0.136800	0.224033	0.253400	0.250300	...	-12.758419	1.398038	
1	0.094800	0.159800	0.224500	0.256033	0.256167	...	-10.091735	1.573857	
2	0.095100	0.136300	0.239200	0.272267	0.272800	...	-8.192064	2.322434	
3	0.070267	0.133900	0.223500	0.260400	0.264833	...	-11.195216	1.315719	
4	0.078367	0.132867	0.234000	0.268400	0.264000	...	-11.755173	1.440045	

	predSOC	aspect_sichei	slope_sichei	curvature_sichei	model_twi	srtm_30	\
0	9.9325	116.565048	2.078012	-0.105323	7.437472	1560.0	
1	12.5035	45.000000	2.627810	-0.000000	6.509323	1562.0	
2	9.4790	30.963757	2.708566	-0.000000	8.963918	1556.0	
3	16.8675	90.000000	1.858794	-0.000000	7.954509	1544.0	
4	15.0005	57.528809	3.027705	0.105323	6.367439	1542.0	

	rivers_distance_sichei	roads_distances_sichei
0	990.223694	126.301727
1	921.653076	82.789207
2	722.070557	57.369698
3	346.116974	0.000000
4	292.126770	131.545120

```
[5 rows x 31 columns]
```

```
[ ]: # print(soc_utm.isnull().sum())  
point_data = soc_utm.dropna() # For now, let's drop rows with any NaN values
```

```
<IPython.core.display.HTML object>
```

geometry	0
B11	0
B12	0
B2	0
B3	0
B4	0
B5	0
B6	0

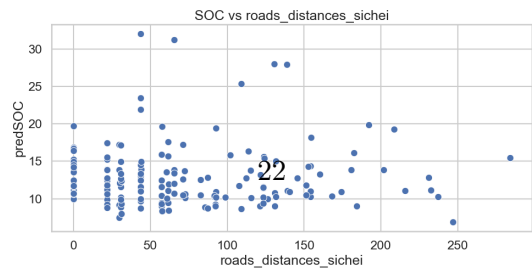
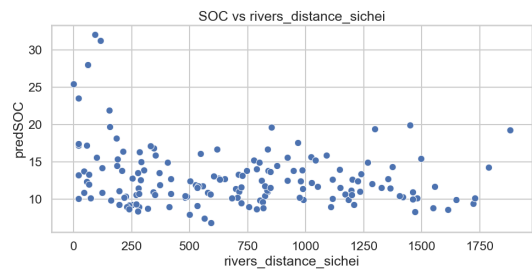
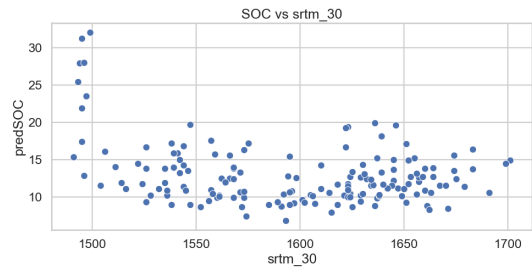
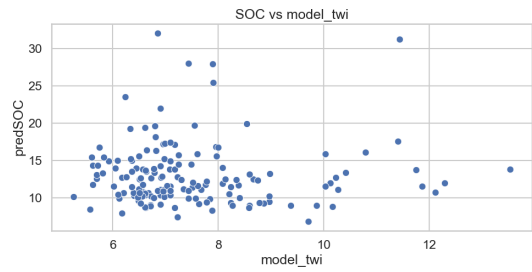
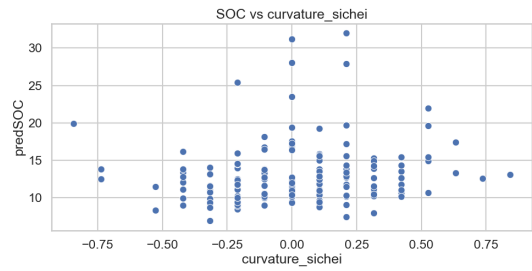
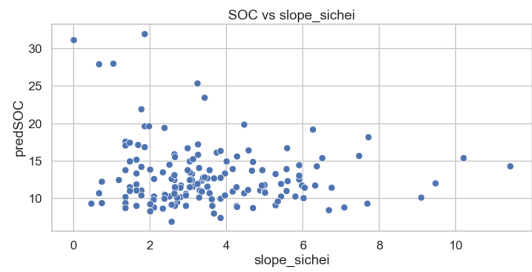
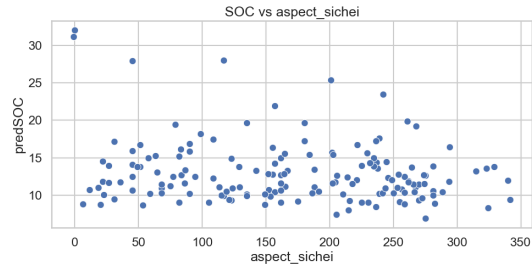
B7	0
B8	0
B8A	0
B9	0
BSI	0
CPR	0
NBR	0
NDI	0
NDSI	0
NDVI	0
RVI	0
SAVI	0
VH	0
VV	0
VVVHR	0
predSOC	0
aspect_sichei	0
slope_sichei	0
curvature_sichei	0
model_twi	0
srtm_30	0
rivers_distance_sichei	0
roads_distances_sichei	0

dtype: int64

Distribution Visualization

```
[74]: fig, axes = plt.subplots(nrows=len(raster_files), ncols=1, figsize=(10, 5 *
    ↪len(raster_files)))
    for i, column in enumerate(raster_files):
        get_first_name = lambda path: os.path.splitext(os.path.basename(path))[0]
        col_name = get_first_name(column)
        sns.scatterplot(x=point_data[col_name], y=point_data['predSOC'],
    ↪ax=axes[i]) # Replace 'your_soc_column'
        axes[i].set_title(f'SOC vs {col_name}')
    plt.tight_layout()
    plt.show()
```

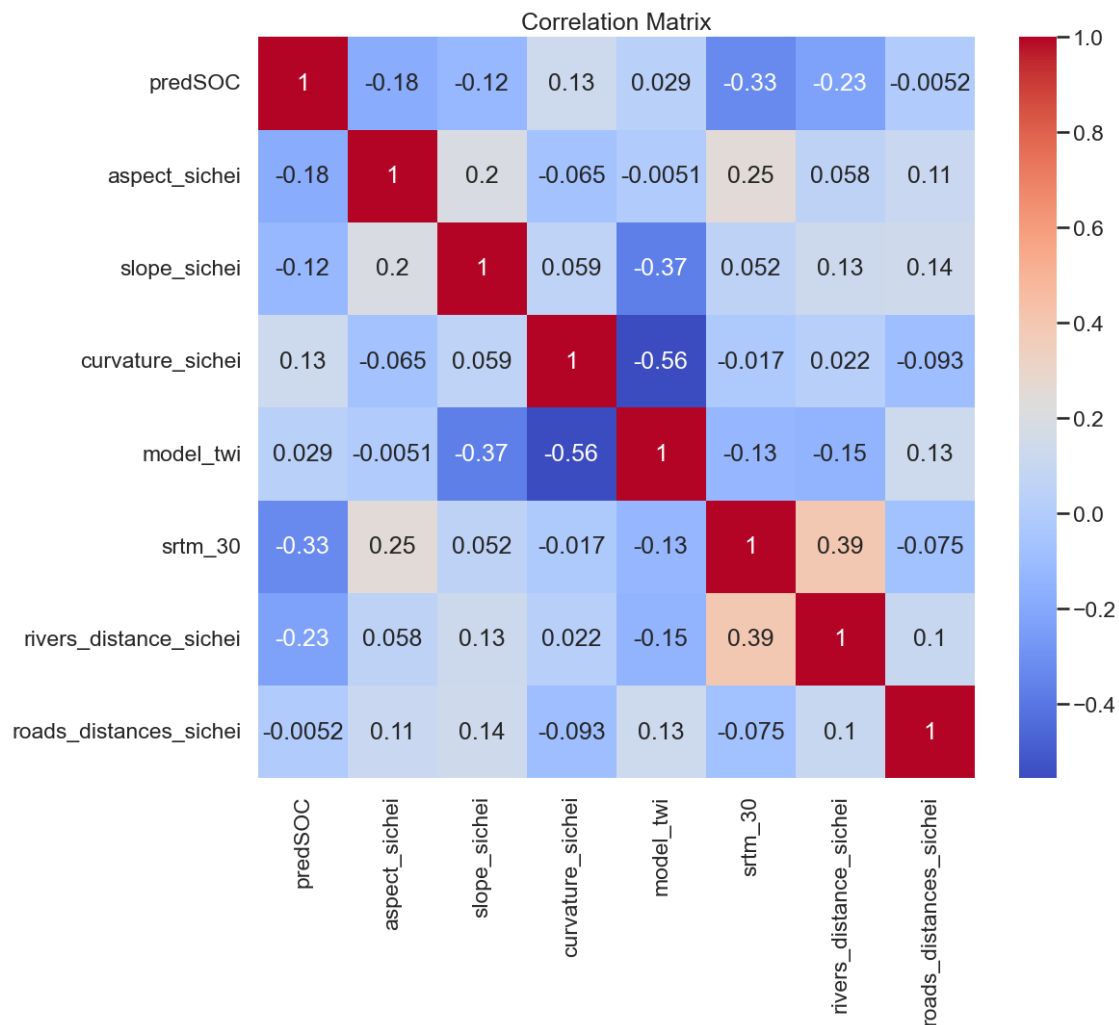
<IPython.core.display.HTML object>



Correlation

```
[75]: correlation_matrix = point_data[['predSOC'] + [get_first_name(col) for col in raster_files]].corr() # Replace 'your_soc_column'
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

<IPython.core.display.HTML object>



3.1 Modeling

```
[82]: X = point_data[[get_first_name(col) for col in raster_files]]
      y = point_data['predSOC'] # Replace 'your_soc_column'
```

<IPython.core.display.HTML object>

```
[83]: from sklearn.ensemble import RandomForestRegressor
      from sklearn.feature_selection import RFE
      from sklearn.model_selection import train_test_split

      # Split data into training and testing sets
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
      ↪random_state=42)

      # RFE with Random Forest
      rf_model = RandomForestRegressor(random_state=42)
      rfe_rf = RFE(estimator=rf_model, n_features_to_select=16) # Adjust the number
      ↪of features to select
      rfe_rf.fit(X_train, y_train)
      print("RFE (Random Forest) Selected Features:", list(X_train.columns[rfe_rf.
      ↪support_]))

      # RFE with XGBoost
      from xgboost import XGBRegressor
      xgb_model = XGBRegressor(random_state=42)
      rfe_xgb = RFE(estimator=xgb_model, n_features_to_select=16) # Adjust the number
      ↪of features to select
      rfe_xgb.fit(X_train, y_train)
      print("RFE (XGBoost) Selected Features:", list(X_train.columns[rfe_xgb.
      ↪support_]))
```

<IPython.core.display.HTML object>

```
RFE (Random Forest) Selected Features: ['aspect_sichei', 'slope_sichei',
'curvature_sichei', 'model_twi', 'srtm_30', 'rivers_distance_sichei',
'roads_distances_sichei']
RFE (XGBoost) Selected Features: ['aspect_sichei', 'slope_sichei',
'curvature_sichei', 'model_twi', 'srtm_30', 'rivers_distance_sichei',
'roads_distances_sichei']
```

```
[84]: from sklearn.model_selection import GridSearchCV
      from sklearn.metrics import mean_squared_error, r2_score

      # Define parameter grids for Random Forest
      param_grid_rf = {
          'n_estimators': [100, 200, 300],
          'max_depth': [None, 5, 10],
          'min_samples_split': [2, 5, 10],
```

```

    'min_samples_leaf': [1, 3, 5]
}

# Grid Search for Random Forest
grid_search_rf = GridSearchCV(estimator=RandomForestRegressor(random_state=42),
                              param_grid=param_grid_rf,
                              cv=5, # 5-fold cross-validation
                              scoring='neg_mean_squared_error',
                              n_jobs=-1)
grid_search_rf.fit(X_train[X_train.columns[rfe_rf.support_]], y_train) # Use
↳selected features
best_rf_model = grid_search_rf.best_estimator_
print("Best Random Forest Model:", best_rf_model)

# Define parameter grids for XGBoost
param_grid_xgb = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}

# Grid Search for XGBoost
grid_search_xgb = GridSearchCV(estimator=XGBRegressor(random_state=42),
                                param_grid=param_grid_xgb,
                                cv=5,
                                scoring='neg_mean_squared_error',
                                n_jobs=-1)
grid_search_xgb.fit(X_train[X_train.columns[rfe_xgb.support_]], y_train) # Use
↳selected features
best_xgb_model = grid_search_xgb.best_estimator_
print("Best XGBoost Model:", best_xgb_model)

```

<IPython.core.display.HTML object>

Best Random Forest Model: RandomForestRegressor(max_depth=5, n_estimators=300, random_state=42)

Best XGBoost Model: XGBRegressor(base_score=None, booster=None, callbacks=None, colsample_bylevel=None, colsample_bynode=None, colsample_bytree=1.0, device=None, early_stopping_rounds=None, enable_categorical=False, eval_metric=None, feature_types=None, feature_weights=None, gamma=None, grow_policy=None, importance_type=None, interaction_constraints=None, learning_rate=0.1, max_bin=None, max_cat_threshold=None, max_cat_to_onehot=None, max_delta_step=None, max_depth=3, max_leaves=None, min_child_weight=None, missing=nan, monotone_constraints=None, multi_strategy=None, n_estimators=200,

```
n_jobs=None, num_parallel_tree=None, ...)
```

```
[85]: # Evaluate Random Forest
y_pred_rf = best_rf_model.predict(X_test[X_test.columns[rfe_rf.support_]])
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2_rf = r2_score(y_test, y_pred_rf)
print(f"Random Forest RMSE on Test Set: {rmse_rf:.2f}")
print(f"Random Forest R-squared on Test Set: {r2_rf:.2f}")

# Evaluate XGBoost
y_pred_xgb = best_xgb_model.predict(X_test[X_test.columns[rfe_xgb.support_]])
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))
r2_xgb = r2_score(y_test, y_pred_xgb)
print(f"XGBoost RMSE on Test Set: {rmse_xgb:.2f}")
print(f"XGBoost R-squared on Test Set: {r2_xgb:.2f}")
```

<IPython.core.display.HTML object>

Random Forest RMSE on Test Set: 3.35
Random Forest R-squared on Test Set: 0.48
XGBoost RMSE on Test Set: 3.40
XGBoost R-squared on Test Set: 0.47

```
[86]: def predict_soc_raster(model, raster_files, output_path, profile,
    ↪selected_features):
    # Read all raster data into a stack
    raster_data = []
    for file in raster_files:
        with rasterio.open(file) as src:
            raster_data.append(src.read(1))

    # Get the dimensions
    num_rows, num_cols = raster_data[0].shape

    # Initialize an empty array for predictions
    predictions = np.zeros((num_rows, num_cols))

    # Iterate through each cell
    for i in range(num_rows):
        for j in range(num_cols):
            cell_values = [raster[i, j] for raster in raster_data]
            # Create a dictionary of feature values for the current cell
            cell_features = {raster_files[k].replace('.tif', ''):
    ↪cell_values[k] for k in range(len(raster_files))}
            # Select only the features used in the model
            prediction_input = pd.DataFrame([cell_features])[selected_features]
            predictions[i, j] = model.predict(prediction_input)[0]
```

```

# Write the predictions to a new raster file
with rasterio.open(output_path, 'w', **profile) as dst:
    dst.write(predictions, 1)

# Get the profile from one of your input rasters
with rasterio.open(raster_files[0]) as src:
    profile = src.profile

output_prediction_path = './exports/predicted_soc.tif'
# Assuming best_rf_model and the features selected by RFE with RF
predict_soc_raster(best_rf_model, raster_files, output_prediction_path,
    profile, list(X_train.columns[rfe_rf.support_]))
print(f"Soil organic carbon prediction raster saved to: {output_prediction_path}")

# You would do a similar process if XGBoost was your chosen model

```

<IPython.core.display.HTML object>

```

-----
KeyError                                Traceback (most recent call last)
Cell In[86], line 34
    32 output_prediction_path = './exports/predicted_soc.tif'
    33 # Assuming best_rf_model and the features selected by RFE with RF
--> 34
    predict_soc_raster(best_rf_model, raster_files, output_prediction_path, profile, list(X_train.columns[rfe_rf.support_]))
    35 print(f"Soil organic carbon prediction raster saved to: {output_prediction_path}")
    37 # You would do a similar process if XGBoost was your chosen model

Cell In[86], line 21, in predict_soc_raster(model, raster_files, output_path, profile, selected_features)
    19 cell_features = {raster_files[k].replace('.tif', ''): cell_values[k] for k in range(len(raster_files))}
    20 # Select only the features used in the model
--> 21 prediction_input = pd.DataFrame([cell_features])[selected_features]
    22 predictions[i, j] = model.predict(prediction_input)[0]
    24 # Write the predictions to a new raster file

File c:\Users\roywaswa\AppData\Roaming\mamba\envs\dsa\Lib\site-packages\pandas\core\frame.py:4108, in DataFrame.__getitem__(self, key)
    4106     if is_iterator(key):
    4107         key = list(key)
-> 4108     indexer = self.columns._get_indexer_strict(key, "columns")[1]
    4110 # take() does not accept boolean indexers
    4111 if getattr(indexer, "dtype", None) == bool:

```

```

File c:
↳ \Users\roywaswa\AppData\Roaming\mamba\envs\dsa\Lib\site-packages\pandas\core\indexes\base.
↳ py:6200, in Index._get_indexer_strict(self, key, axis_name)
    6197 else:
    6198     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
-> 6200 self._raise_if_missing(keyarr, indexer, axis_name)
    6202 keyarr = self.take(indexer)
    6203 if isinstance(key, Index):
    6204     # GH 42790 - Preserve name from an Index

```

```

File c:
↳ \Users\roywaswa\AppData\Roaming\mamba\envs\dsa\Lib\site-packages\pandas\core\indexes\base.
↳ py:6249, in Index._raise_if_missing(self, key, indexer, axis_name)
    6247 if nmissing:
    6248     if nmissing == len(indexer):
-> 6249         raise KeyError(f"None of [{key}] are in the [{axis_name}]")
    6251     not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].
↳ unique())
    6252     raise KeyError(f"{not_found} not in index")

```

```

KeyError: "None of [Index(['aspect_sichei', 'slope_sichei', 'curvature_sichei',
↳ 'model_twi',\n          'srtm_30', 'rivers_distance_sichei',\n
↳ 'roads_distances_sichei'],\n          dtype='object')] are in the [columns]"

```